

SnapBack（拉回你的注意力）PRD v0.2.1

英文名：SnapBack

中文名：拉回你的注意力

Slogan：Pull your code back on track.

产品形态：IDE 插件 + 终端 CLI + Codex / Claude Code 可调用的 Skill / MCP Server

版本定位：在 v0.2 基础上做实现对齐补丁；继续采用 v0.2 的 contract 字段体系，不采用 v0.3/v0.4 的字段命名改动。

0. v0.2.1 统一结论

本版本只解决 v0.2 中影响 Codex 开发的一组规格缺口：

1. **提问与 contract 收敛**：定义 deterministic contract generation 与最多 3 个问题的关系，明确何时问、是否阻塞、用户不回答怎么办。
2. **职责泄漏检测**：定义 layer preset + 项目扫描 + snapback.config.json 的优先级，给出可实现规则。
3. **任务生命周期与 patch churn**：定义 task_id、active task、region key、audit history 与关闭任务。
4. **提醒交互差异**：IDE 主提醒只有 2 个动作；CLI 可以有 3 个短选项；MCP 只返回结构化结果。
5. **仓库结构与命令**：补齐 schemas/ 目录；PRD 与 implementation prompt 均包含 snapback audit。

v0.2.1 的核心原则：**MVP 先用规则和项目扫描实现，不依赖 LLM**；后续可以把 **contract generator** 替换成 **LLM provider**，但接口不变。

1. 核心判断

SnapBack 服务的不是“前端开发”，而是所有由 AI agent 执行的 coding 项目。前端登录弹窗只是示例，产品本质是：

1. 把用户模糊的自然语言需求转成可执行的代码契约；
2. 让 agent 在写代码前明确边界、接口、模块职责和验收标准；
3. 在项目开始堆补丁、职责混乱、偏离主线时，低打扰地把用户和 agent 拉回正轨。

SnapBack 不直接替代 Cursor、Codex、Claude Code、Cline、Continue、Aider。它是这些 coding agent 的增强层：写之前定契约，写之中守边界，写之后给验收。

2. 核心痛点

痛点 A：自然语言不能稳定转成代码级实现

小白用户经常只会描述“我要什么效果”，但不会说明应该改哪些文件、哪些模块不应该动、接口怎么设计、哪些边界条件必须处理，以及怎么判断功能完成。

结果是：agent 很快写出“看起来能跑”的代码，但用户不知道工程是怎么来的，也不知道后续怎么维护。

痛点 B：长任务 vibe coding 会失控成代码泥潭

非一次性项目里，如果用户缺少工程意识，持续让 agent “再补一个功能”“再修一下 bug”，很容易出现：文件越来越长、逻辑混在一起、同一块代码反复打补丁、fallback/try-catch 越堆越多、prompt/tool/memory/schema 混在一个 agent 文件里、当前实现逐渐偏离最初需求。

SnapBack 要在项目“还能救”的时候提醒，而不是等到用户已经完全改不动。

3. 产品目标与非目标

P0 目标

1. 生成 Requirement-to-Code Contract (R2C Contract)：把自然语言需求转成代码层可执行契约。
2. 给 agent 提供可读的开发上下文：目标、边界、接口、模块职责、实现步骤、验收流程。
3. 跨技术栈检测 drift / sludge：不限前端，至少支持通用代码项目、JS/TS、Python 的基础检测。
4. 主动提醒但不打扰：提醒必须具体、可执行、能解释原因，不能泛泛输出“建议优化代码”。
5. 所有结构性改动都必须用户确认，不默认自动改代码。

非目标

- 不做完整 IDE；
 - 不做重型项目管理平台；
 - 不做通用代码审查 SaaS；
 - 不追求一次性解决所有语言的深度静态分析；
 - 不把每一个 lint 问题都弹窗；
 - 不在用户未确认前自动重构代码。
-

4. 产品形态

4.1 IDE 插件

MVP 先做 VS Code 插件，后续扩展 JetBrains。插件不是控制台，而是嵌在开发流里的轻量守护层：

- Status Bar：显示 SnapBack: On Track / Drift / Sludge Risk；
- Command Palette：Start Task、Show Contract、Run Guard、Review Fix Plan；
- Diagnostics：把风险标在具体文件、函数、模块位置；
- Webview / Side Panel：只在用户点击 Review Plan 后展示详细原因、修改方案和验收步骤；
- 后续宠物形象：低打扰提醒入口，不常驻打断。

4.2 CLI

给 Codex、Claude Code、Cline、Aider、Continue CLI 等终端 agent 调用。

```
snapback init
snapback task "add OAuth login"
snapback task "add OAuth login" --non-interactive
snapback task "add OAuth login" --assume-defaults
snapback answer --task <task_id> --question q1 --value "不处理退款"
snapback explain
snapback guard
snapback guard --json
snapback audit
snapback audit --json
snapback patch --plan
snapback handoff
snapback handoff --close
snapback prefs
```

命令区分：

- task：创建任务、扫描项目、生成 draft contract，并根据问题状态进入 needs_user_input、provisional 或 ready。
- answer：写回用户对 open question 的回答，并重新收敛 contract。
- guard：围绕 active task 和当前 git diff 做快速守护，判断是否偏离 contract。
- audit：做更宽的项目健康扫描，可以在没有 active task 时运行，用于发现已有代码泥潭。
- patch --plan：只生成修改计划，不自动应用；如果 task 是 needs_user_input，必须拒绝并提示先回答阻塞问题。
- handoff --close：生成交接摘要并关闭当前任务。

4.3 Agent Skill / MCP Server

SnapBack 暴露为 agent 能调用的工具层：

```
create_task_contract(requirement, mode?)
read_active_contract()
answer_contract_questions(task_id, answers)
audit_drift(task_id?)
audit_sludge(task_id?)
propose_patch_plan(task_id)
generate_acceptance_checklist(task_id)
record_user_preference(preference)
```

Agent 执行约束：

1. 写代码前必须读取或创建 active contract；
 2. 若 contract 状态为 needs_user_input，agent 必须先向用户询问阻塞问题，不能继续生成 patch；
 3. 只能在 contract 范围内修改；
 4. 每完成一个阶段运行 guard；
 5. 若出现 Drift / Sludge Risk，先解释并等待用户确认。
-

5. 核心机制：R2C Contract

R2C Contract 是用户、agent 和代码之间的中间契约。它必须语言无关，能够覆盖前端、后端、脚本、数据分析、Agent 应用、CLI 工具等项目。

5.1 Contract 关键字段

Contract 使用 v0.2 字段命名，但补充 task 状态、生命周期和问题收敛字段：

- schema_version：当前 schema 版本，v0.2.1 固定为 0.2.1。
- task_id：当前任务唯一 ID。
- task_status：draft / needs_user_input / provisional / ready / active / closed。
- raw_user_need：用户原始需求。
- user_goal：转译后的真实目标。
- project_context：自动识别的语言、框架、入口文件、目录结构、测试命令。
- contract_confidence：high / medium / low，由需求清晰度与项目扫描结果决定。
- assumptions：SnapBack 做出的假设，必须可被用户修改。
- open_questions：最多 3 个真正影响代码接口、范围或风险的问题；每个问题包含 blocking、default_assumption 和 status。
- non_goals：明确不做什么，防止过度设计。
- affected_artifacts：预计新增、修改、删除的文件、模块、脚本、接口、配置、prompt、测试。

- `interfaces`: 函数、类、REST API、CLI、schema、event、prompt contract、agent tool 等接口。
- `invariants`: 必须保持不变的业务或工程约束。
- `edge_cases`: 空状态、异常输入、权限、失败、重试、超时、数据缺失等边界。
- `implementation_steps`: 分步骤实现计划。
- `explanation_for_user`: 给小白用户看的“为什么这样拆”。
- `acceptance_criteria`: 功能完成标准。
- `manual_verification`: 用户可一步步执行的人工验收流程。
- `test_plan`: 自动测试或命令行验证计划。
- `rollback_plan`: 出问题如何撤回。
- `lifecycle`: 当前 task 的分支、base commit、创建/更新时间等生命周期元信息。

5.2 open_questions 对象格式

`open_questions` 不再只是字符串，而是结构化问题对象：

```
{
  "id": "q1",
  "question": "取消订单是否需要同时触发退款？",
  "why_it_matters": "这会决定是否改支付/退款模块，属于高风险跨模块范围。",
  "blocking": true,
  "default_assumption": "本任务不处理退款，只实现取消订单状态变更。",
  "options": ["不处理退款", "同步触发退款", "只预留退款 hook"],
  "affects": ["non_goals", "affected_artifacts", "interfaces"],
  "status": "unanswered"
}
```

6. 如何提问并收敛 Contract

6.1 和 deterministic heuristics 的关系

MVP 的 contract generation 使用 deterministic heuristics，并不代表“不问问题”。含义是：

- 用规则和项目扫描生成 contract；
- 用规则和项目扫描判断需求是否模糊；
- 用规则生成最多 3 个 open questions；
- 后续可替换为 LLM provider，但输入输出必须保持同一个 contract schema。

6.2 触发提问的条件

SnapBack 不是每次都问问题。只有满足以下任一条件时才生成 `open_questions`：

- 需求中缺少目标对象，例如“给系统加登录”，但项目里同时存在 admin、customer、internal 三套登录入口；
- 需求可能触碰高风险模块，例如支付、权限、数据库迁移、删除数据、隐私数据、生产配置；
- 同一个需求存在多个合理实现路径，且路径会影响接口、数据结构或文件结构；
- 用户要求“顺便重构/顺便优化/全部改一下”，但没有明确范围；
- project scanner 找到多个候选入口，无法确定应该改哪个。

6.3 阻塞规则

open question 分为两类：

- blocking = true：没有用户回答就不能安全写代码。例如是否修改数据库 schema、是否接入支付/退款、是否改变权限模型、是否删除/迁移数据。
- blocking = false：SnapBack 有安全默认假设。用户不回答时，可以按 default_assumption 继续，但 contract 状态标记为 provisional。

6.4 用户不回答怎么办

- IDE 交互：展示最多 3 个问题；如果用户关闭问题面板，阻塞问题保持 needs_user_input，非阻塞问题使用默认假设进入 provisional。
- CLI 交互式：逐个询问阻塞问题；用户可输入答案，也可输入 assume 接受默认假设。
- CLI --assume-defaults：所有非阻塞问题使用默认假设；若存在阻塞问题，仍然返回 needs_user_input。
- CLI --non-interactive / MCP：不直接卡死进程，返回结构化结果：task_status = needs_user_input、exit_code = 2、questions 列表。agent 必须把问题转述给用户。
- 对于 needs_user_input 的 contract，SnapBack 允许 explain/guard/audit，但不允许 patch --plan 生成修改方案。

6.5 Contract 收敛流程

1. snapback task 创建 draft contract。
 2. 生成最多 3 个 open questions。
 3. 如果没有 blocking 问题：
 - 没有任何问题：task_status = ready；
 - 只有非阻塞问题且用户未答：套用默认假设，task_status = provisional。
 4. 如果存在 blocking 问题：task_status = needs_user_input，禁止 patch 计划与 agent 修改。
 5. 用户回答后，snapback answer 或 answer_contract_questions() 写答案。
 6. SnapBack 将答案合并到 assumptions/non_goals/affected_artifacts/interfaces/invariants/edge_cases。
 7. 所有 blocking 问题回答后，contract 进入 ready；一旦 agent 开始修改或执行 guard，状态进入 active。
-

7. 主流程

流程 A：NL to Code 精确落地

1. 用户在 IDE 或终端输入自然语言需求。
2. SnapBack 扫描项目结构，识别语言、框架、入口、测试方式、已有架构习惯。
3. 使用 deterministic heuristics 生成 draft contract。
4. 根据需求清晰度与项目扫描结果，进入 ready / provisional / needs_user_input。
5. 如果需要提问，先收敛 contract；阻塞问题未回答前不生成 patch。
6. agent 按 contract 实现，而不是自由发挥。
7. SnapBack 对 git diff 做 guard：是否改了不该改的模块，是否漏掉边界，是否破坏架构职责。
8. 输出小白可读解释：每个模块为什么存在、做了什么、不做什么、如何验收。

流程 B：主动拉回主线

触发条件：

- 保存文件后风险分数跨过阈值；
- agent 完成一个阶段后运行 snapback guard；
- 用户手动运行 snapback audit；
- MCP tool 被 agent 调用。

提醒必须包含：问题在哪里、为什么这是偏离或泥潭信号、建议怎么改、改动影响哪些文件、用户如何人工验收。

8. 职责泄漏检测：可实现规则

8.1 规则来源优先级

职责泄漏检测的规则来源按以下优先级合并：

1. **最高优先级：snapback.config.json**。用户显式配置的 layer rules、阈值、accepted patterns 覆盖默认规则。
2. **中优先级：项目自动识别 preset**。根据目录、依赖、文件名、入口文件推断项目类型，例如 frontend、backend、python_script、data_pipeline、rag_agent、cli。
3. **最低优先级：generic 默认规则**。所有项目都启用文件长度、函数长度、重复代码、patch churn、contract drift、fallback smell 等通用规则。

8.2 项目类型识别

scanner 使用以下信号识别项目类型：

- package.json 依赖：React/Next/Vite -> frontend；Express/Fastify/Nest -> backend；commander/yargs -> CLI。

- Python 文件: pyproject.toml、requirements.txt、notebooks/、scripts/、pipelines/。
- 目录名: components/pages/app/routes/controllers/services/repositories/models/schemas/agents/rag/tools/prompts/memory/retrievers/pipelines/cli。
- 文件名: route.ts、controller.ts、service.ts、repository.ts、agent.py、retriever.py、prompt.ts、cli.ts 等。

8.3 Layer rule 数据结构

```
{
  "id": "backend-route-no-db-write",
  "preset": "backend",
  "layer": "route",
  "where": ["routes/**", "src/routes/**", "controllers/**"],
  "allowed_responsibilities": ["request_parse", "auth_check", "response_format"],
  "forbidden_responsibilities": ["database_write", "state_machine",
    "heavy_domain_logic"],
  "forbidden_imports": ["@/repositories/**", "@/db/**"],
  "forbidden_patterns": ["SELECT ", "INSERT ", "UPDATE ", "DELETE ", "prisma.",
    "mongoose.", "sequelize."],
  "forbidden_cooccurrence": [],
  "message": "Route/controller should not own database writes or a full business state
    machine. Move it into service/repository layers."
}
```

8.4 Responsibility tag

Analyzer 给文件或函数打 responsibility tags。MVP 内置 tags:

- ui_rendering
- network_io
- auth_session
- cache_storage
- request_parse
- response_format
- schema_validation
- domain_logic
- state_machine
- database_read
- database_write
- file_io
- data_cleaning
- modeling

- plotting
- cli_parsing
- filesystem_side_effect
- prompt_authoring
- tool_execution
- memory_management
- retrieval
- fallback_handling
- final_answer_generation

8.5 判定逻辑

出现以下情况之一，即可输出 responsibility leak finding：

1. 文件/函数命中当前 layer 的 forbidden_responsibilities；
2. 文件/函数命中 forbidden_imports 或 forbidden_patterns；
3. 文件/函数同时拥有 3 个以上高层责任，且不在 accepted_patterns 中；
4. 命中 forbidden_cooccurrence，例如 agent 文件同时出现 prompt/tool/memory/retrieval/fallback/final answer；
5. git diff 将某个 artifact 的责任扩展到 contract 未声明的范围。

每条 finding 必须包含：rule_id、preset_source、具体位置、证据、建议修复、影响范围、人工验收步骤。

9. Patch churn 与任务生命周期

9.1 task_id 生成

task_id 格式：

YYYYMMDD-HH:mm:ss-<slug>-<short_hash>

示例：20260118-143012-add-order-cancel-a1b2c3。

生成规则：

- slug 从用户需求中提取 3-6 个英文/拼音关键词，无法提取时用 task；
- short_hash = sha1(raw_user_need + base_commit + created_at).slice(0, 6)；
- task_id 创建后不可变。

9.2 本地任务目录

snapback init 创建 .snapback/。snapback task 创建任务目录：

```
.snapback/  
  active-task.json  
  mission.md  
  architecture.md  
  decisions.md  
  user-prefs.json  
  accepted-patterns.json  
  tasks/  
    <task_id>/  
      contract.json  
      audit-history.jsonl  
      snapshots.jsonl  
      acceptance.md  
      handoff.md
```

active-task.json 只保存当前 active task 的 task_id 和路径。

9.3 任务开始与结束

- 创建：snapback task 生成 contract，并记录 base_commit、branch、created_at、working_tree_dirty。
- 激活：任务进入 ready/provisional 后，一旦执行 guard 或 agent 产生相关 diff，状态变为 active。
- 暂停：用户可保留 active task，不关闭，但 Later 只在当前 task 内抑制提醒。
- 关闭：snapback handoff --close 写入 handoff.md，记录 final diff summary，并将 contract 状态改为 closed。

9.4 region 归一化

Patch churn 不是按绝对行号判断，而是按 region key 判断。

Region key 优先级：

1. AST symbol：normalized_file_path#symbol_kind:qualified_name，例如 src/orders/service.ts#function:cancelOrder。
2. 若没有 AST：使用 diff hunk 的上下文签名 normalized_file_path#hunk:<context_hash>。
3. 若文件重命名：优先使用 git rename detection 映射旧路径；失败时视为新 region。
4. 若 branch/rebase 改变 base commit：继续使用 symbol name + context hash；置信度不足时 finding 降级为 warning，并在 evidence 中注明 base_commit_changed。

9.5 patch churn 判定

在同一个 active task 内，如果同一 region 被修改 3 次及以上，触发 patch churn warning。若同时伴随 fallback smell 或 responsibility leak，升级为 critical。

audit-history.jsonl 每行记录:

```
{
  "timestamp": "2026-01-18T14:42:01Z",
  "task_id": "20260118-143012-add-order-cancel-a1b2c3",
  "branch": "feature/order-cancel",
  "base_commit": "abc123",
  "head_commit": "def456",
  "command": "guard",
  "region_changes": [
    {
      "region_key": "src/orders/service.ts#function:cancelOrder",
      "file_path": "src/orders/service.ts",
      "start_line": 20,
      "end_line": 88,
      "change_type": "modify"
    }
  ],
  "findings": []
}
```

10. 提醒交互设计

10.1 IDE 主提醒：只有两个动作

主动提醒只出现两个按钮:

- **Review Plan**: 查看原因、修正计划、影响范围、验收步骤;
- **Later**: 本次先不处理, 任务内不重复提醒, 除非风险升级。

Generate Patch、Apply Patch、Mark as Acceptable Pattern、Tune Rule 只出现在详情页中, 不出现在 IDE 主提醒里。

10.2 CLI 允许三个短选项

CLI 没有 Webview 详情页, 所以 warning 后允许三个短选项:

[R] Review plan [L] Later [M] Mark acceptable for this task

这不是和 IDE 冲突, 而是终端交互的降噪处理: M 只作用于当前 task, 写入 task-scoped accepted pattern, 不写入全局偏好, 除非用户显式运行 snapback prefs。

10.3 MCP 无 UI 动作

MCP server 不返回按钮，只返回结构化状态：ok / drift / sludge_risk / needs_user_input, 以及 findings、questions 或 patch_plan。由上层 agent 决定如何转述给用户。

11. 仓库结构与交付物

11.1 Monorepo 结构

```
snapback/  
  package.json  
  pnpm-workspace.yaml  
  tsconfig.base.json  
  schemas/  
    task-contract.v0.2.1.schema.json  
  packages/  
    shared/          # types, zod schemas, constants, schema loader  
    src/  
      schemas/  
      types/  
    core/            # contract engine, scanners, analyzers, risk scoring  
    cli/             # snapback CLI  
    extension-vscode/ # VS Code extension  
    mcp-server/      # MCP tools  
  analyzers/  
    generic/  
    js-ts/  
    python/  
  skills/  
    codex/  
      AGENTS.md  
    claude/  
      SKILL.md  
  templates/  
    snapback.config.example.json  
  examples/  
    nextjs-demo/  
    express-api-demo/  
    python-csv-demo/  
    rag-agent-demo/
```

11.2 必须交付

- schemas/task-contract.v0.2.1.schema.json

- packages/shared 中的 TypeScript/Zod schema
 - snapback init/task/answer/explain/guard/audit/patch --plan/handoff/prefs
 - .snapback/tasks/<task_id>/ 生命周期存储
 - generic、JS/TS、Python 三类 analyzer adapter
 - VS Code 插件基础能力
 - MCP server tools
 - 4 个 demo fixture 和测试
-

12. 第一阶段开发范围

第一阶段目标：做出一个真的能被 Codex / Claude Code 调用、也能装进 VS Code 的可用 MVP。

优先级：

1. 建立 pnpm monorepo；
2. 定义 v0.2.1 TaskContract schema；
3. 实现 snapback init/task/answer/explain/guard/audit/patch --plan/handoff/prefs；
4. 实现 question gating：blocking 问题阻塞 patch，非阻塞问题可走 provisional；
5. 实现 task lifecycle 与 .snapback/tasks/<task_id>/ 存储；
6. 实现 generic analyzer；
7. 实现 JS/TS analyzer；
8. 实现 Python analyzer 的基础 LOC、函数长度、import、pattern 检测；
9. 实现 responsibility leak 的 layer preset + config override；
10. 实现 patch churn 的 region key 与 audit history；
11. 实现 VS Code status bar + command palette + Review Plan webview；
12. 实现低打扰通知：IDE 只有 Review Plan / Later；
13. 实现 MCP server tools；
14. 加入 4 个 demo 仓库：前端、后端、Python CSV、RAG agent。

最终效果：用户仍然是在 vibe coding，但 agent 不再乱扩写、乱补丁；小白用户也能知道每一步代码为什么这样落地。